第2课：训练深度网络

- 李伟昌, weichangli@zju.edu.cn
- 浙江大学**海洋人工智能**课程讲义

- 版本 1, 2026年5月20日

时长： 45分钟
目标： 掌握反向传播、优化和正则化技术

2.1 回顾与动机

上节课我们学习了：

- 前向传播计算预测
- 损失函数衡量预测质量

关键问题： 我们如何学习权重$\mathbf{W}$和偏置$\mathbf{b}$?

学习问题

给定：

- 训练数据：$\{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^n$
- 模型：$\hat{\mathbf{y}} = f(\mathbf{x}; \boldsymbol{\theta})$ 其中$\boldsymbol{\theta} = \{\mathbf{W}, \mathbf{b}\}$
- 损失函数：$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})$

求：$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \frac{1}{n}\sum_{i=1}^n \mathcal{L}(f(\mathbf{x}^{(i)}; \boldsymbol{\theta}), \mathbf{y}^{(i)})$

2.2 反向传播：核心算法

直观理解

反向传播 = 链式法则 + 动态规划

我们需要每层$\frac{\partial \mathcal{L}}{\partial \mathbf{W}}$来更新权重。

链式法则回顾

对于复合函数$f(g(x))$：

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g} \cdot \frac{\partial g}{\partial x}$$

单层反向传播

考虑单层：$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$，$\mathbf{a} = f(\mathbf{z})$

前向：

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

$$\mathbf{a} = f(\mathbf{z})$$

反向： 我们从下一层接收到$\frac{\partial \mathcal{L}}{\partial \mathbf{a}}$。

通过激活的梯度：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}} \odot f'(\mathbf{z})$$

参数梯度：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}} \cdot \mathbf{x}^T$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}}$$

传回的梯度：

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \mathbf{W}^T \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{z}}$$

完整反向传播算法

```
1. 前向传播：计算并存储所有a[l], z[l]
2. 初始化：δ[L] = ∇_a[L] L ⊙ f'(z[L])
3. 对于 l = L-1 到 1:
   a. δ[l] = (W[l+1]^T · δ[l+1]) ⊙ f'(z[l])
   b. ∂L/∂W[l] = δ[l] · a[l-1]^T
   c. ∂L/∂b[l] = δ[l]
4. 更新：W[l] ← W[l] - α · ∂L/∂W[l]
```

计算复杂度

- 前向传播：O(参数数量)
- 反向传播：O(参数数量)
- 反向传播在网络规模上是**线性**的（高效！）

可视化演练：逐步反向传播

受HMKCODE的反向传播可视化和Karpathy的micrograd启发

考虑一个具有一个隐藏层的简单网络：

```
前向传播（我们计算的）：
================================

x₁=2 ──w₁=0.11──┐
                ├──→ h₁=0.368 ──w₅=0.14──┐
x₂=3 ──w₂=0.21──┘                        ├──→ ŷ=0.191
                ├──→ h₂=0.432 ──w₆=0.15──┘
x₁=2 ──w₃=0.12──┤
                │
x₂=3 ──w₄=0.08──┘

目标：y = 1
误差：L = ½(y - ŷ)² = ½(1 - 0.191)² = 0.327
```

反向传播（计算梯度）：

链式法则告诉我们误差如何通过每个操作反向流动：

```
∂L/∂w₆ = ∂L/∂ŷ · ∂ŷ/∂z₃ · ∂z₃/∂w₆
       = (ŷ - y)   · ŷ(1-ŷ)   · h₂
       = -0.809    · 0.155    · 0.432
       = -0.054

∂L/∂w₅ = (ŷ - y) · ŷ(1-ŷ) · h₁ = -0.046
```

梯度流可视化：

```
反向传播：
=============

     ┌────────────────────────────────────────┐
     │  dL/dŷ = (ŷ - y) = -0.809              │
     └────────────────────────────────────────┘
                        │
     ┌──────────────────┴────────────────────┐
     │  dL/dz₃ = (ŷ - y)·σ'(z₃) = -0.125     │  (σ' = sigmoid导数)
     └───────────┬──────────────┬────────────┘
                 │              │
          ┌──────┘              └──────┐
          │                            │
    ┌─────▼──────┐              ┌──────▼─────┐
    │ dL/dw₅     │              │ dL/dw₆     │
    │ = dL/dz₃·h₁│              │ = dL/dz₃·h₂│
    │ = -0.046   │              │ = -0.054   │
    └────────────┘              └────────────┘
          │                            │
          └────────────┬───────────────┘
                       │
                ┌──────▼──────┐
                │ dL/dh₁      │
                │ = dL/dz₃·w₅ │
                │ = -0.0175   │
                └─────────────┘
                       │
                ┌──────▼──────┐
                │ dL/dz₁      │
                │ = dL/dh₁·   │
                │   ReLU'(z₁) │
                │ = -0.0175   │
                └─────────────┘
                       │
                ┌──────▼──────┐
                │ dL/dw₁      │
                │ = dL/dz₁·x₁ │
                │ = -0.035    │
                └─────────────┘
```

权重更新：

```
w₁_new = w₁ - η·(dL/dw₁) = 0.11 - 0.5·(-0.035) = 0.1275
w₆_new = w₆ - η·(dL/dw₆) = 0.15 - 0.5·(-0.054) = 0.177
```

示例：2层网络反向传播（代码）

```
import numpy as np

def backward_pass(X, y, cache, params):
    """
    cache包含前向传播的z1, a1, z2, a2
    params包含W1, b1, W2, b2
    """
    m = X.shape[0]
    z1, a1, z2, a2 = cache
    W1, b1, W2, b2 = params

    # 输出层梯度
    dz2 = a2 - y  # softmax + 交叉熵
    dW2 = (1/m) * np.dot(a1.T, dz2)
    db2 = (1/m) * np.sum(dz2, axis=0)

    # 隐藏层梯度
    da1 = np.dot(dz2, W2.T)
    dz1 = da1 * (a1 > 0)  # ReLU导数
    dW1 = (1/m) * np.dot(X.T, dz1)
    db1 = (1/m) * np.sum(dz1, axis=0)

    return dW1, db1, dW2, db2
```

2.3 梯度下降变体

普通梯度下降

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t)$$

问题： 完整数据集梯度计算昂贵。

随机梯度下降（SGD）

在单个样本上更新：

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t; \mathbf{x}^{(i)}, \mathbf{y}^{(i)})$$

优点： 快速迭代，逃离浅层局部最小值
缺点： 高方差，嘈杂收敛

小批量梯度下降

在大小为B的批次上更新：

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \cdot \frac{1}{B} \sum_{i=1}^{B} \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t; \mathbf{x}^{(i)}, \mathbf{y}^{(i)})$$

最佳点： B = 32-512（取决于GPU内存）

带动量的SGD

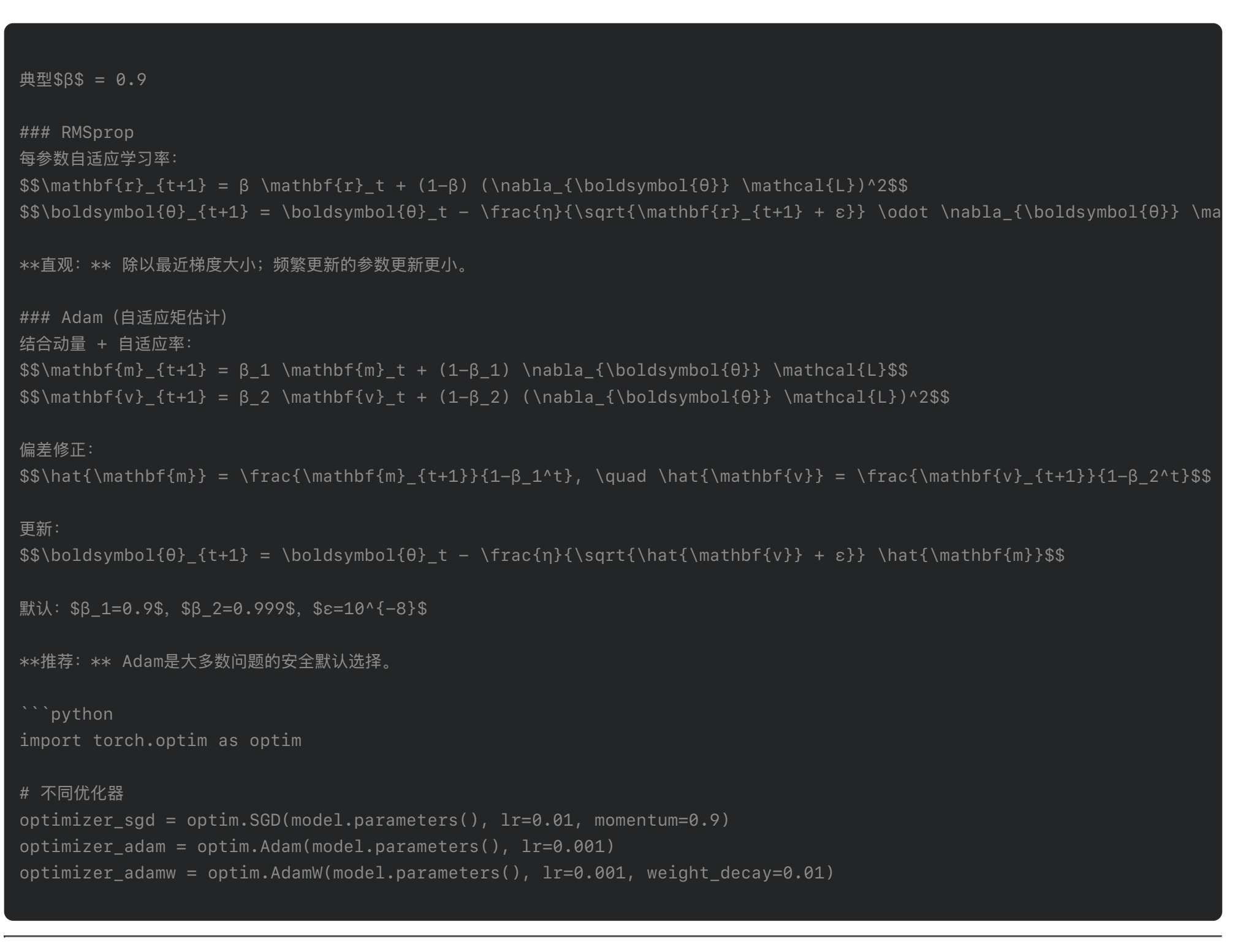
累积速度：

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t + \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \mathbf{v}_{t+1}$$

直观： 球滚下山；抑制振荡，在一致方向上加速。

视觉对比：



Momentum accumulates velocity to dampen gradient oscillations.
Standard SGD
Momentum
Accumulate velocity (typically β = 0.9):
$\mathbf{v}_{t+1} = \beta \mathbf{v}_t + \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_t)$
Apply update:
$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \mathbf{v}_{t+1}$

速度项累积一致的梯度方向， 帮助逃离浅层局部最小值和平坦区域。

典型$β$ = 0.9

### RMSprop
每参数自适应学习率:
$$\mathbf{r}_{t+1} = β \mathbf{r}_t + (1-β) (\nabla_{\boldsymbol{θ}} \mathcal{L})^2$$
$$\boldsymbol{θ}_{t+1} = \boldsymbol{θ}_t - \frac{η}{\sqrt{\mathbf{r}_{t+1} + ε}} \odot \nabla_{\boldsymbol{θ}} \ma

直观： 除以最近梯度大小；频繁更新的参数更新更小。

### Adam (自适应矩估计)
结合动量 + 自适应率:
$$\mathbf{m}_{t+1} = β_1 \mathbf{m}_t + (1-β_1) \nabla_{\boldsymbol{θ}} \mathcal{L}$$
$$\mathbf{v}_{t+1} = β_2 \mathbf{v}_t + (1-β_2) (\nabla_{\boldsymbol{θ}} \mathcal{L})^2$$

偏差修正:
$$\hat{\mathbf{m}} = \frac{\mathbf{m}_{t+1}}{1-β_1^t}, \quad \hat{\mathbf{v}} = \frac{\mathbf{v}_{t+1}}{1-β_2^t}$$

更新:
$$\boldsymbol{θ}_{t+1} = \boldsymbol{θ}_t - \frac{η}{\sqrt{\hat{\mathbf{v}} + ε}} \hat{\mathbf{m}}$$

默认：$β_1=0.9$, $β_2=0.999$, $ε=10^{-8}$

推荐： Adam是大多数问题的安全默认选择。

```python
import torch.optim as optim

# 不同优化器
optimizer_sgd = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
optimizer_adam = optim.Adam(model.parameters(), lr=0.001)
optimizer_adamw = optim.AdamW(model.parameters(), lr=0.001, weight_decay=0.01)
```

2.4 学习率调度

为什么需要调度?

- 初始高LR：快速进展
- 后期低LR：微调、收敛

常见调度策略

阶梯衰减：

$$\eta_t = \eta_0 \cdot \gamma^{\lfloor t/\text{epoch}_{\text{drop}} \rfloor}$$

指数衰减：

$$\eta_t = \eta_0 \cdot e^{-kt}$$

余弦退火：

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\frac{t}{T}\pi))$$

预热（对transformers至关重要）： 在前N步线性从0增加到目标LR。

```
from torch.optim.lr_scheduler import StepLR, CosineAnnealingLR

scheduler = StepLR(optimizer, step_size=30, gamma=0.1)
scheduler = CosineAnnealingLR(optimizer, T_max=200)

# 在训练循环中
optimizer.step()
scheduler.step()
```

2.5 初始化

为什么初始化很重要

- 太大：激活爆炸
- 太小：激活消失
- 刚好：稳定的前向/反向信号

Xavier/Glorot初始化

用于tanh/sigmoid激活：

$$W_{ij} \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}}\right)$$

或正态变体：

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{in} + n_{out}}\right)$$

启发式： 保持各层激活方差。

He初始化

用于ReLU激活：

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{in}}\right)$$

考虑ReLU杀死一半激活（零均值）。

```
import torch.nn as nn

# PyTorch默认对Linear/Conv层使用Kaiming (He)
layer = nn.Linear(784, 256)
nn.init.xavier_uniform_(layer.weight)  # Glorot
nn.init.kaiming_normal_(layer.weight, nonlinearity='relu')  # He
```

2.6 正则化

为什么需要正则化?

深度网络有数百万参数 → 高过拟合风险。

L2正则化（权重衰减）

向损失添加惩罚：

$$\mathcal{L}_{reg} = \mathcal{L} + \lambda \sum_{\mathbf{W}} ||\mathbf{W}||_F^2$$

更新规则：

$$\mathbf{W} \leftarrow (1 - \eta\lambda)\mathbf{W} - \eta \nabla_{\mathbf{W}} \mathcal{L}$$

注意： 使用AdamW（解耦权重衰减）而不是Adam + L2惩罚。

Dropout

训练期间：以概率p随机将神经元置零。

```
layer = nn.Dropout(p=0.5)  # 50% dropout
```

为什么有效：

- 集成解释：训练指数级多个精简网络
- 防止神经元共适应

测试时： 使用所有神经元（无dropout），或在训练期间使用反转dropout（缩放1/p，测试时无缩放）。

数据增强

转换训练数据以增加有效数据集大小：

图像：

- 随机裁剪/翻转
- 旋转、缩放
- 颜色抖动
- Cutout/随机擦除

文本：

- 同义词替换
- 回译
- 随机插入/删除

早停

监控验证损失；停止改善时停止训练。

```
# 简单早停逻辑
best_val_loss = float('inf')
patience = 10
epochs_no_improve = 0

for epoch in range(max_epochs):
    train(model)
    val_loss = validate(model)

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        torch.save(model.state_dict(), 'best_model.pth')
        epochs_no_improve = 0
    else:
        epochs_no_improve += 1
        if epochs_no_improve >= patience:
            print("触发早停")
            break
```

批归一化

归一化批次内的激活：

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$y_i = \gamma \hat{x}_i + \beta$$

好处：

- 减少内部协变量偏移
- 允许更高学习率
- 充当正则化（批次统计的噪声）
- 使训练更稳定

```
# 典型ConvNet块
block = nn.Sequential(
    nn.Conv2d(in_ch, out_ch, 3, padding=1),
    nn.BatchNorm2d(out_ch),
    nn.ReLU(),
    nn.Dropout(0.3)
)
```

2.7 训练技巧和调试

训练清单

1. **从小开始：** 首先过拟合单个批次
2. **验证初始化时的损失：** 随机猜测应给出预期损失
3. **检查梯度：** 它们的大小合理吗?
4. **监控指标：** 损失、准确率、学习率
5. **可视化：** 权重、激活、预测

常见问题

症状	可能原因	解决方案
损失不下降	LR太低，反向传播有bug	增加LR，检查梯度
损失爆炸（NaN）	LR太高，梯度爆炸	梯度裁剪，降低LR
训练OK，验证差	过拟合	更多正则化，更多数据
卡在平台期	糟糕的初始化，ReLU死亡	更好的初始化，leaky ReLU

梯度检查

```
# 数值梯度检查（仅用于调试）
def grad_check(model, x, y, epsilon=1e-7):
    model.zero_grad()
    loss = criterion(model(x), y)
    loss.backward()

    for name, param in model.named_parameters():
        if param.grad is not None:
            grad_approx = (loss(x+epsilon) - loss(x-epsilon)) / (2*epsilon)
            diff = torch.abs(param.grad - grad_approx)
            print(f"{name}: max diff = {diff.max()}")
```

2.8 实验：训练深度网络

练习1：实现反向传播

完成上节课的NumPy反向传播实现。用数值近似验证梯度。

练习2：比较优化器

用以下方法训练相同网络：

- SGD（无动量）
- 带动量的SGD
- Adam

比较收敛速度和最终性能。

练习3：正则化实验

在容易过拟合的数据集上训练（例如，有限数据的CIFAR-10）：

- 基线（无正则化）
- 添加dropout
- 添加权重衰减
- 添加数据增强
- 组合所有

测量每个的训练/验证差距。

关键要点

1. 反向传播使用链式法则高效计算梯度
2. Adam是大多数问题的良好默认优化器；最终微调用SGD+动量
3. 学习率调度显著影响收敛
4. 适当的初始化防止信号消失/爆炸
5. 正则化（dropout、权重衰减、增强）防止过拟合
6. 批归一化稳定和加速训练
7. 总是首先用小规模测试验证你的实现

延伸阅读

核心材料

1. CS231n Optimization Notes
2. Why Momentum Really Works - Distill.pub
3. How Does Batch Normalization Help Optimization?
4. AdamW Paper

可视化学习资源

5. **HMKCODE的Backpropagation Step by Step** - 本讲中的逐步数值示例和视觉梯度流图受此教程启发。强烈推荐手工完成具体的反向传播计算。

6. **Andrej Karpathy的Neural Networks: Zero to Hero** - 特别是第2讲（"The spelled-out intro to backpropagation and training neural networks"），它从头开始构建micrograd。这些笔记中的计算图可视化遵循Karpathy的操作追踪梯度方法。

7. **Karpathy's Micrograd** - "一个微小的Autograd引擎（有料！）：在动态构建的DAG上实现反向传播（反向模式自动微分），并在其上构建了一个小神经网络库，带有类似PyTorch的API。"阅读约100行的micrograd是真正理解自动微分的最佳方式。